

Beyond MPTRANS

Fused integer-only attention and weight-traffic-aware co-design for short-sequence edge transformers

Research proposal and roadmap · starting point: the MPTRANS paper and its RTL (24 Verilog files)

THE PROPOSAL IN ONE MINUTE	
The setting	Voice assistants classify short commands (8–32 tokens). MPTRANS is a RISC-V chip with a DistilBERT accelerator built for exactly this job.
The problem	At this length the chip spends its time and energy fetching 42.5 MB of weights from memory, not on attention. MPTRANS puts its precision effort into attention, and the shared code is not yet a finished, balanced design.
The idea	Move less data and share it: (A) all-integer attention, (B) weights packed at 8/4/2 bits and unpacked on the fly, (C) one weight fetch serving several requests at once.
What to expect	Energy per request about 5–7× lower in a first-order model (robust). Speed-up is conditional on the real compute peak and on how often requests overlap. Accuracy at low bit-width must be shown by training, not assumed.

1. What exists today

MPTRANS is an edge accelerator for transformer text classification. A RISC-V processor (scalar core plus vector unit) is tightly coupled to a DistilBERT block with 6 layers that are reused one after another. The target workload is intent recognition (Amazon MASSIVE) for smart-home and assistant devices.

Its main design choices

- **Precision per step.** 8-bit integers for Query × Key, 16-bit floating point (FP16) for softmax, 4-bit integers for Attention × Value. Quantization-aware training (QAT) recovers the accuracy lost to the lower precision.
- **Custom instructions.** Five new RISC-V instructions let the core load tokens, start inference, poll status, read the result and reset the accelerator.
- **Reported results.** 94.1% accuracy on MASSIVE; 846,846 cycles for an 8-token input; ASIC synthesis (14 nm) of 3.28 mm² and 8.4 mW at 820 MHz; an FPGA build on the ZCU104 board.

What the shared RTL contains

24 Verilog files, about 4,000 lines: top level and SoC (2), attention (5), feed-forward network and GELU (2), arithmetic units (5), LayerNorm, residual, classifier and main controller (4), RISC-V core and custom-instruction decoder (2), memories and DMA (2), shared definitions (2). There are no testbenches, synthesis scripts or reports in the zip.

WHY THIS MATTERS FOR THE PROPOSAL

A proposal to “improve MPTRANS” is only credible if the starting point is measured, not quoted. So the roadmap begins by making the baseline reproducible (Phase 0) and only then builds the three mechanisms on top of it.

Quick glossary

S	number of tokens (word pieces) in one input	Roofline	a chart showing whether a chip is limited by memory speed or by compute speed
Weights	the model's learned numbers; 42.5 MB in the 6 layers, fetched from memory every run	QAT	quantization-aware training: retraining while simulating low precision so the model adapts
INT8 / INT4 / FP16	8-bit integer, 4-bit integer, 16-bit floating point; fewer bits means smaller and cheaper but less precise	Softmax	the attention step that turns scores into probabilities
GOPS	billions of arithmetic operations per second	DRAM	the external main memory; moving a byte from it costs far more energy than computing on it

2. The problem, in plain words

2.1 Problem 1: the chip waits for its weights

Picture a chef who walks to a distant warehouse for every ingredient, uses it 8 times, then throws it away. A short sentence is exactly that: every weight fetched from memory is used only S times (S = number of tokens). Work per byte moved is therefore tiny, about $2 \times S = 16$ operations per byte at $S = 8$.

With an assumed edge memory of 12.8 GB/s, that ratio caps the chip near **205 GOPS**, and just moving the 42.5 MB of weights takes **3.3 ms** per run. Meanwhile self-attention is only about 0.2% of the arithmetic at $S = 8$, yet it receives all of MPTRANS's precision effort.

2.2 Problem 2: the float detour in softmax

Softmax runs in FP16, so values are converted from INT8 to FP16 and back, and the chip carries floating-point adders and multipliers just for this step. It is a real inefficiency, but at short sequences it is a small slice of the work: worth removing for a clean all-integer chip, not a big speed lever.

2.3 What the RTL review found

Reading the shared code against the paper turned up the points below. They are static observations, not simulation results, so each one should be confirmed in Phase 0.

What the code shows	Why it matters
Linear layers do one multiply-add per clock (acc += x*w loops in <code>multi_head_attention.v</code> and <code>feed_forward.v</code>); NUM_PES = 8 is defined but not used there.	Counting loop iterations gives about 340 M cycles at $S = 8$, against 846,846 reported ($\approx 400\times$ apart). Needs a simulation to reconcile.
Each multiply reads a 64-bit weight word but keeps 8 bits.	Up to $8\times$ wasted on-chip read bandwidth.
Top level is not fully wired: weight-memory write ports and DMA start are tied to 0, embedding finishes instantly, many module outputs are left open.	Weight loading and end-to-end accuracy cannot be reproduced from this zip alone.
Six weight banks are declared as 8 MB block RAM each (≈ 48 MB).	The paper reports 336 BRAMs (≈ 1.5 MB). Real weights must stream from DRAM, which is the premise of this proposal.
Softmax "piecewise exp" returns one constant per segment; instruction names differ (<code>nlp.result</code> / <code>nlp.config</code> vs <code>nlp.read</code> / <code>nlp.reset</code>).	Approximation quality and paper-code consistency need checking.
No synthesis scripts or reports are included.	The 3.28 mm ² , 8.4 mW and 453 GOPS figures cannot be reproduced from the zip; treat them as unverified.

Consequence for the plan. At 8 multipliers $\times$ 820 MHz the real peak would be about **13 GOPS**, so the code as written is limited by compute, not memory. The memory-wall argument applies to a properly sized compute array. Building that array is therefore step one of the roadmap, before weight traffic can be the bottleneck.

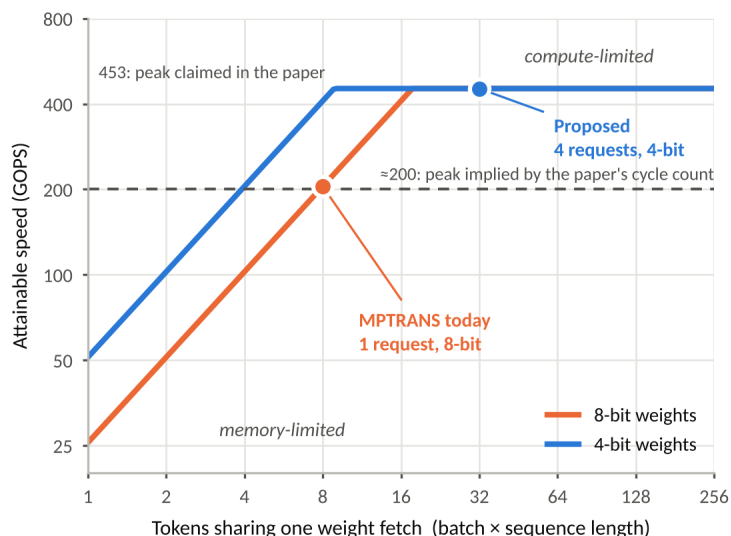
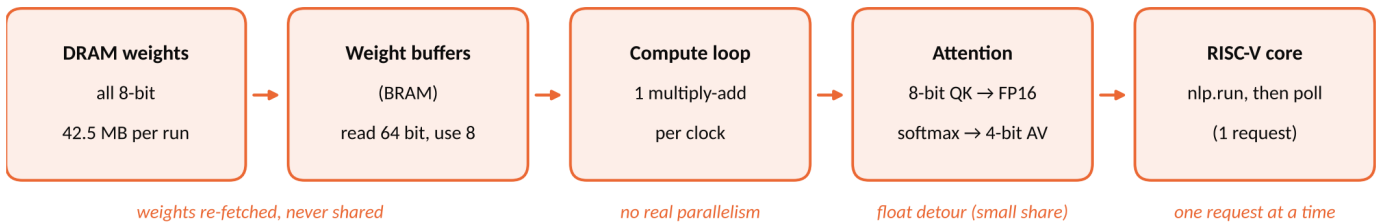


Figure 1. Speed limits. The rising part is memory-limited, the flat part compute-limited. Sharing each weight fetch among more tokens, or storing weights in fewer bits, moves the chip up the slope. Bandwidth 12.8 GB/s is an assumption.

3. The proposed fix: three mechanisms

The redesign keeps MPTRANS's RISC-V host and custom-instruction idea, but changes the data path so that fewer bytes move and every byte does more work.

TODAY (MPTRANS, as shared)



PROPOSED

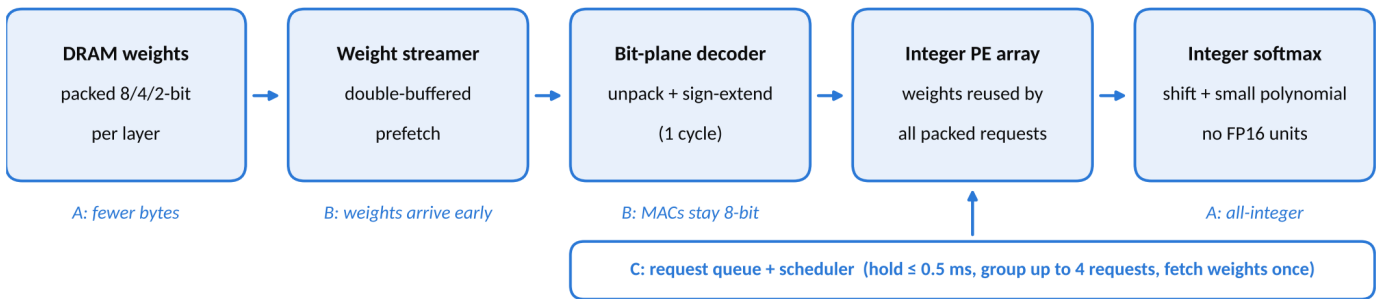


Figure 2. Today versus proposed data path. Letters A, B and C mark the three mechanisms.

A • Integer-only softmax How it works. Subtract the row maximum, turn e^x into 2^x , then split the result into a whole part (a bit-shift) and a small fraction (a 2-term polynomial). This follows the I-BERT method. GELU and LayerNorm are made integer too. Why it helps. Removes the FP16 adder, multiplier and conversions. One uniform integer data path. Watch out. Attention is a small share at $S = 8$, so expect area and simplicity gains more than speed.	B • Packed mixed-precision weights How it works. A compiler picks 8, 4 or 2 bits per layer from its sensitivity (Q-BERT / HAQ style) and QAT retrain the model. Weights are packed with a 2-bit header. A decoder at the array entrance unpacks and sign-extends them to 8 bits. Why it helps. Bytes moved fall in proportion to bit-width ($2\times$ at 4-bit). The multipliers stay unchanged. Watch out. 2-bit layers can cost accuracy. DRAM burst alignment erodes part of the gain. The decoder adds a little area.	C • Request-packing scheduler How it works. Hold the first request for a short window (about 0.5 ms), group up to 4 requests, fetch each weight once and apply it to all of them. A command queue replaces the “run, then poll” interface. Why it helps. Weight traffic per request is divided by the group size. Watch out. Adds up to the window in delay, and only pays off when requests really overlap (a hub serving many devices, not one speaker).
---	---	---

How they fit together. B and C attack the same bottleneck. Their gains multiply for memory traffic, but stop adding once the compute array becomes the limit (the flat part of Figure 1). The best pair of (bit-width, group size) therefore moves with sequence length S . Mapping that moving optimum is the research question, and it is a better story than three separate tricks.

Left out on purpose. The idea of morphing on-chip SRAM into a KV-cache for $S \geq 128$ is moved to future work. It sits outside the $S = 8\text{--}32$ niche and would dilute the contribution.

4. Expected effects, and what we cannot claim yet

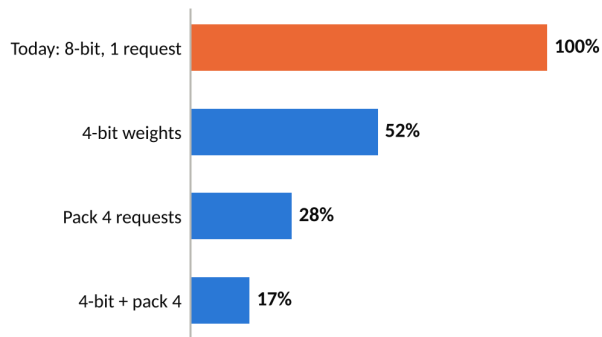


Figure 3. Relative energy per request at $S = 8$. First-order model: DRAM 5 pJ/bit, 0.25 pJ per 8-bit multiply-add, 12.8 GB/s. Moving weights from DRAM is 76–96% of the energy across a 2–15 pJ/bit range, so the result is robust.

What the model says

We wrote a small analytical model (*roofline_model.py*, delivered with this proposal) that combines DistilBERT's size with weight bit-width, group size, compute peak, memory bandwidth and Poisson request arrivals.

Energy is the strong result. It barely depends on the compute peak, because it is dominated by DRAM traffic. The benefit shrinks as sequences grow: 6.0× at $S = 8$, 3.7× at $S = 32$, 1.9× at $S = 128$ (4-bit weights, 4-request group).

Speed is conditional. If the real compute peak is near 200 GOPS, neither mechanism raises throughput at $S = 8$. At 453 GOPS it improves 2.2× at $S = 8$ and 1.1× at $S = 16$.

Metric	Expectation	Confidence
Energy per request	1.9× lower with 4-bit weights, 3.5× with 4-request groups, 6.0× with both ($S = 8$; 4.7–7.2× across DRAM-cost assumptions)	High
Energy vs sequence length	Gain falls from 6.0× ($S = 8$) to 3.7× ($S = 32$) and 1.9× ($S = 128$); shows where the optimum moves	High
Throughput	Up to 2.2× at $S = 8$ if the effective peak is about 453 GOPS; none if about 200 GOPS; after the array is widened it follows memory bandwidth	Conditional
Latency of one request	Adds up to the hold window (0.5 ms). At 1–10 requests/s groups almost never form, so the cost is pure delay; benefit appears at hundreds of requests/s	Medium
Area and power	FP16 units go away; decoder, queue and larger buffers are added. Net effect roughly neutral until synthesized	Low
Accuracy	Target within 1 point of the FP baseline on MASSIVE, with 2-bit layers used sparingly. Cannot be shown analytically	Unknown

Honest limits. These numbers come from a first-order model, not from measurements. Its inputs (bandwidth, energy per access, compute peak) are assumptions that Timeloop/Accelerly and a real memory model must replace. Accuracy, area and power need training runs and synthesis. The earlier “8.5× less traffic, 0.4 ms floor” figure is an *amortized* bound (8-bit to 4-bit gives 2×, a group of 4 gives 4×), not a per-request latency, and should be quoted that way.

What would make the claims strong

- Run MPTRANS and the redesign through the **same** model or synthesis flow, so every comparison is like for like.
- Ablation: bit-width only, grouping only, integer softmax only, and all combined, over $S = 8$ –128.
- An arrival-rate study with realistic traffic (single device versus hub) so the grouping delay is reported honestly.

5. Roadmap: from the MPTRANS RTL to the finished design

Phase	What we do	RTL files touched	Done when
0 • Trust the baseline weeks 1–2	Add a testbench and simulate (Verilator or Icarus). Wire the top level (weight loading, activation buffers). Build a bit-exact Python reference. Reconcile the cycle count. Run synthesis to get real area and power.	<i>distilbert_top, dma_controller, memory_blocks</i>	RTL matches Python on MASSIVE samples; cycles and area reproduce.
1 • Model and profile weeks 2–4	Extend our roofline model into Timeloop/Accelergy. Measure stalls versus busy cycles. Fix targets: array size, buffer sizes, group size.	<i>none (model only)</i>	Baseline energy and latency sweep over S, bits, group size.
2 • Real compute array weeks 4–6	Replace the one-multiply-add loops with a tiled, weight-stationary array. Match its width to memory bandwidth.	<i>feed_forward, multi_head_attention, attention_head, mac_unit_int8</i>	Cycles within about 10% of the model.
3 • Mechanism A weeks 6–8	Write an integer softmax (shift plus polynomial); make GELU and LayerNorm integer; delete the FP16 units. Put it inside QAT.	<i>softmax_fp16 → softmax_int; gelu_approx; layer_norm; fp16_* removed</i>	Accuracy within 0.5 point of the float baseline; FP16 area gone.
4 • Mechanism B weeks 8–11	Compiler script: layer sensitivity to bit-widths and packed layout. Add weight streamer with double buffering and the bit-plane decoder. QAT at 8/4/2 bits.	<i>new_weight_streamer, bitplane_decoder; memory_blocks</i>	DRAM bytes track average bit-width; accuracy-versus-bits curve.
5 • Mechanism C weeks 11–14	Add a command queue and scheduler FSM; extend the instruction decoder with enqueue and status; make the array batch-aware.	<i>transformer_fsm, nlp_isa_decoder, riscv_nlp_soc</i>	Throughput and latency under Poisson arrivals match the model.
6 • Evaluate and write weeks 14–18	Full sweeps (S 8–128, 2–8 bits, group sizes), ablations, arrival-rate study, MobileBERT and TinyBERT too. Optional FPGA prototype of decoder plus scheduler. Write the paper.	—	Figures, tables and a paper draft.

Decision gate after Phase 1. If profiling shows the design is compute-bound even with a sensibly sized array, the story shifts to array-bandwidth co-design; the same model and sweeps still apply.

6. Risks and answers

- **“Weight quantization is crowded.”** The contribution is the joint (bit-width, group size, S) analysis and the scheduler, not a new compression scheme.
- **“Batching hurts latency.”** We report latency and energy separately and study single-device versus hub traffic.
- **“The baseline is not credible.”** We build our own verified baseline and run both designs through the same flow.

7. Where to submit

Good fits: DATE, ASP-DAC, ISLPED, ICCD, ISVLSI (conferences); IEEE TVLSI and TCAS-I (journals). A short version could go to IEEE Computer Architecture Letters. Check the current deadlines before planning.

Key references (confirm details before citing): I-BERT, Kim et al., ICML 2021 • Q-BERT, Shen et al., AAAI 2020 • HAQ, Wang et al., CVPR 2019 • GOBO, Zadeh et al., MICRO 2020 • FQ-BERT, Liu et al., DATE 2021 • Roofline, Williams et al., CACM 2009 • Timeloop, Parashar et al., ISPASS 2019 • Accelergy, Wu et al., ICCAD 2019 • MPTRANS paper and RTL (as supplied).